

ПРАКТИЧЕСКАЯ РАБОТА №5

НЕДЕЛЯ 5

JWT-аутентификация: регистрация, вход и защищённые маршруты

Дисциплина: Веб-программирование на JavaScript / Node.js

Формат: индивидуальная практическая работа на 1 неделю

Цель: научиться реализовывать регистрацию и авторизацию пользователей в REST API на Node.js/Express, безопасно хранить пароли с bcrypt, выдавать JWT-токены и защищать маршруты с помощью middleware.

Продолжение проекта: работа выполняется на основе практической работы №4. В проекте уже должны использоваться Express, MongoDB и Mongoose.

Неделя	Основная тема
3	MongoDB Driver + CRUD
4	Mongoose + Schema + Model + Validation
5	JWT + Authentication + Authorization

Рекомендуемое время: 10–14 академических часов

1. Задание студенту

Добавьте в существующий REST API систему пользователей. Пользователь должен иметь возможность зарегистрироваться, войти в систему и получить JWT-токен. Некоторые endpoints проекта должны стать доступными только авторизованным пользователям.

Главная задача недели — понять полный цикл аутентификации: • регистрация

пользователя;

- валидация входных данных;
- хеширование пароля;
- сохранение пользователя в MongoDB через Mongoose;
- login и проверка пароля;
- создание JWT;
- передача JWT через Authorization header;
- проверка токена middleware;
- защита REST endpoints.

2. Архитектура после добавления авторизации

```
project/
  ■■■ config/
  ■■■■ database.js
  ■■■ models/
  ■■■■ Item.js
  ■■■■ User.js
  ■■■ controllers/
  ■■■■ itemController.js
  ■■■■ authController.js
  ■■■ routes/
  ■■■■ itemRoutes.js
  ■■■■ authRoutes.js
  ■■■ middleware/
  ■■■■ authMiddleware.js
  ■■■ errorHandler.js
  ■■■ .env
  ■■■ .env.example
  ■■■ .gitignore
  ■■■ app.js
  ■■■ server.js
  ■■■ package.json
```

3. Установка необходимых пакетов

```
npm install bcryptjs jsonwebtoken
```

Используйте `bcryptjs` для хеширования паролей и `jsonwebtoken` для создания и проверки JWT. Важно: пароль пользователя никогда не должен храниться в MongoDB в открытом виде.

4. Модель User

Создайте `models/User.js`. Минимально пользователь должен содержать `email` и `password`. Рекомендуется также добавить `name` и `role`.

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true,
      minlength: 2,
      maxlength: 50
    },

    email: {
      type: String,

      required: true,
      unique: true,
      lowercase: true,
```

```

    trim: true
  },

  password: {
    type: String,
    required: true,
    minlength: 6,
    select: false
  },

  role: {
    type: String,
    enum: ['user', 'admin'],
    default: 'user'
  },
},
{ timestamps: true }
);

module.exports = mongoose.model('User', userSchema);

```

`select: false` позволяет не возвращать `password` в обычных запросах к `User`. При `login` пароль можно запросить отдельно через `select('+password')`.

5. Регистрация пользователя

Создайте endpoint: `POST /api/auth/register`.

Тело запроса:

```

{
  "name": "Aruzhan",
  "email": "aruzhan@example.com",
  "password": "12345678"
}

```

Алгоритм регистрации:

- проверить обязательные поля;
- привести `email` к единому формату;
- проверить, существует ли пользователь с таким `email`;
- хешировать пароль через `bcrypt`;
- создать `User`;
- вернуть безопасные данные пользователя без `password`.

При успешной регистрации рекомендуется вернуть `201 Created`.

6. Хеширование пароля

Пароль должен быть преобразован в хеш до сохранения.

```

const bcrypt = require('bcryptjs');

const hashedPassword = await bcrypt.hash(password, 10);

```

Число 10 — количество `salt rounds`. В учебной работе этого достаточно. Важно понимать, что `bcrypt` не предназначен для расшифровки пароля: при `login` выполняется сравнение введенного пароля с сохранённым хешем.

```

const isPasswordCorrect = await bcrypt.compare(
  password,
  user.password
);

```

7. Login

Создайте endpoint: `POST /api/auth/login`.

```

{
  "email": "aruzhan@example.com",
  "password": "12345678"
}

```

Алгоритм:

- найти пользователя по email;
- получить password через select('+password');
- сравнить пароль через bcrypt.compare();
- если пароль неправильный — вернуть 401 Unauthorized;
- если всё корректно — создать JWT.

8. Создание JWT

```
const jwt = require('jsonwebtoken');

const token = jwt.sign(
  {
    userId: user._id,
    role: user.role
  },
  process.env.JWT_SECRET,
  {
    expiresIn: '1d'
  }
);
```

JWT_SECRET должен находиться в .env и не должен попадать в Git.

```
JWT_SECRET=your_super_secret_key
JWT_EXPIRES_IN=1d
```

Успешный login должен вернуть токен:

```
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "user": {
    "id": "...",
    "name": "Aruzhan",
    "email": "aruzhan@example.com",
    "role": "user"
  }
}
```

9. Что такое JWT

JWT (JSON Web Token) — компактный токен, который позволяет серверу проверить, что запрос связан с ранее успешно аутентифицированным пользователем.

Упрощённо JWT состоит из трёх частей:

- Header — информация о типе токена и алгоритме;
- Payload — данные/claims, например userId и role;
- Signature — подпись, позволяющая проверить целостность токена.

Важно: JWT payload не следует считать секретным хранилищем. Не помещайте туда пароль, банковские данные или другие секреты.

10. Middleware проверки JWT

Создайте middleware/authMiddleware.js. Он должен извлекать токен из заголовка:

```
Authorization: Bearer <JWT_TOKEN>
```

Алгоритм middleware:

- получить Authorization header;
- проверить наличие Bearer token;

- выполнить jwt.verify();
- получить userId и role из payload;
- передать данные дальше через req.user;
- при отсутствии/ошибке токена вернуть 401.

```
const authMiddleware = (req, res, next) => {
  const authHeader = req.headers.authorization;
```

```

if (!authHeader || !authHeader.startsWith('Bearer ')) {
  return res.status(401).json({
    message: 'Authorization token required'
  });
}

const token = authHeader.split(' ')[1];

try {
  const decoded = jwt.verify(
    token,
    process.env.JWT_SECRET
  );

  req.user = decoded;
  next();
} catch (error) {
  return res.status(401).json({
    message: 'Invalid or expired token'
  });
}
};

```

11. Защита существующих маршрутов

Выберите минимум 2 endpoint из проекта недели 4 и сделайте их доступными только авторизованным пользователям.

Например:

```

router.get('/', authMiddleware, getItems);
router.post('/', authMiddleware, createItem);
router.put('/:id', authMiddleware, updateItem);
router.delete('/:id', authMiddleware, deleteItem);

```

GET /api/auth/register и POST /api/auth/login защищать JWT middleware не нужно — иначе пользователь не сможет получить токен.

12. Авторизация по ролям

Добавьте простое разделение ролей: user и admin. Минимально один endpoint должен быть доступен только admin.

```

const adminOnly = (req, res, next) => {
  if (req.user.role !== 'admin') {
    return res.status(403).json({
      message: 'Admin access required'
    });
  }

  next();
};

```

Разница:

- 401 Unauthorized — пользователь не аутентифицирован или токен отсутствует/недействителен.
- 403 Forbidden — пользователь авторизован, но у него недостаточно прав.

13. REST API недели 5

Метод	URL	Доступ	Назначение
POST	/api/auth/register	Public	Регистрация

Практическая работа №5 — JWT Authentication Страница 5

Метод	URL	Доступ	Назначение
POST	/api/auth/login	Public	Вход
GET	/api/auth/me	JWT	Текущий пользователь
GET	/api/items	JWT	Список объектов
POST	/api/items	JWT	Создание

DELETE	/api/items/:id	Admin	Удаление
--------	----------------	-------	----------

14. Endpoint /api/auth/me

Создайте endpoint, который возвращает информацию о текущем пользователе. Он должен быть защищён JWT.

```
GET /api/auth/me
Authorization: Bearer <token>
```

Пример ответа:

```
{
  "id": "665abc...",
  "name": "Aruzhan",
  "email": "aruzhan@example.com",
  "role": "user"
}
```

15. Тестирование в Postman

№	Проверка	Ожидаемый результат
1	Register с корректными данными	201 + user
2	Register с существующим email	400/409
3	Register с коротким паролем	400
4	Login с правильным паролем	200 + JWT
5	Login с неправильным паролем	401
6	GET /api/auth/me без token	401
7	GET /api/auth/me с token	200
8	Защищённый endpoint без token	401
9	Admin endpoint от user	403
10	Admin endpoint от admin	200
11	Истёкший/изменённый token	401

16. Требования к безопасности

- не хранить password в открытом виде;
- не хранить JWT_SECRET в исходном коде;
- добавить .env в .gitignore;
- не возвращать password в ответе API;
- использовать bcrypt для паролей;
- проверять JWT через jwt.verify(), а не просто декодировать payload;
- использовать 401 и 403 по назначению.

17. План работы на 7 дней

День	Задачи
1	Изучить JWT, bcrypt, Authorization header. Установить пакеты.

2	Создать User Schema и регистрацию.
3	Реализовать bcrypt hash и login.
4	Добавить создание JWT и endpoint /api/auth/me.
5	Создать authMiddleware и защитить endpoints.
6	Добавить роли user/admin и проверить 401/403.
7	Postman-тестирование, README, исправление ошибок, демонстрация.

18. Критерии оценивания — 100 баллов

Критерий	Баллы
User Schema + Mongoose	10
Регистрация	15
bcrypt и безопасное хранение пароля	15
Login + JWT	20
JWT middleware	15
Защищённые endpoints	10
Роли и 401/403	5
Postman + README + качество кода	10
Итого	100

19. Контрольные вопросы

- 1. Что такое authentication и чем она отличается от authorization?
- 2. Что такое JWT?
- 3. Из каких частей состоит JWT?
- 4. Что хранится в payload?
- 5. Почему нельзя хранить пароль в JWT?
- 6. Почему JWT_SECRET нельзя публиковать?
- 7. Что делает jwt.sign()?
- 8. Что делает jwt.verify()?
- 9. Почему нельзя проверять токен простым декодированием?
- 10. Что такое Bearer token?
- 11. Как bcrypt хранит пароль?
- 12. Чем bcrypt.compare() отличается от bcrypt.hash()?
- 13. Почему пароль нельзя возвращать в JSON-ответе?
- 14. Что делает select: false в Mongoose?
- 15. Что означает HTTP 401?
- 16. Что означает HTTP 403?
- 17. Как middleware передаёт userId контроллеру?

- 18. Как реализовать роль admin?

• 19. Почему login и register не должны требовать JWT? • 20.

Какие данные нельзя помещать в JWT payload?